

AlphaScalp — Serveur de licence + relais de signaux (MVP bêta)

Brique 1 du moteur. Rôle : recevoir les trades du **bot maître**, les relayer aux **followers** *uniquement si leur clé d'abonnement est active*, et gérer les clés via une page admin. Tout en démo/gratuit pour la bêta — le toggle actif/inactif simule l'état d'abonnement.

Lancer

```
bash

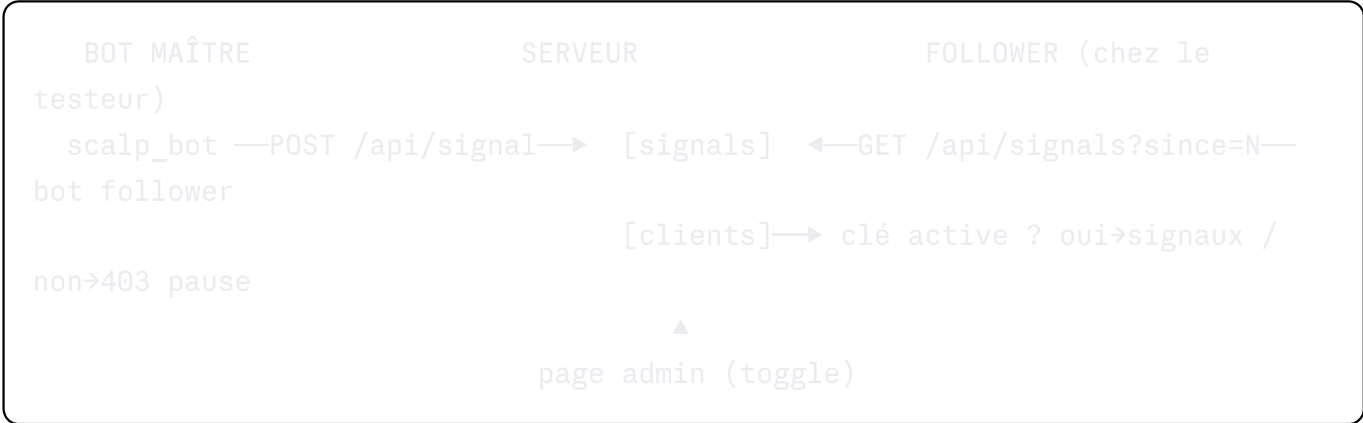
pip install -r requirements.txt

# Jetons (à changer ; sinon valeurs de dev par défaut)
export ALPHASCALP_MASTER_TOKEN="ton-jeton-maitre"
export ALPHASCALP_ADMIN_TOKEN="ton-jeton-admin"

python server.py
```

- Page admin : `http://127.0.0.1:8000/admin?token=<ADMIN_TOKEN>`
- Base SQLite créée automatiquement (`alphascalp.db`).

Flux



Endpoints

Méthode	Route	Auth	Usage
POST	<code>/api/signal</code>	header <code>X-Master-Token</code>	le maître publie un trade (<code>open</code>) (<code>close</code>)
GET	<code>/api/status</code>	header <code>X-API-Key</code>	le follower vérifie son état + dernier id
GET	<code>/api/signals?since=N</code>	header <code>X-API-Key</code>	nouveaux signaux d'id > N (403 si inactif)

Méthode	Route	Auth	Usage
GET	<code>/api/admin/clients</code>	<code>?token=</code>	liste des clés
POST	<code>/api/admin/clients?name=&plan=</code>	<code>?token=</code>	créer une clé
POST	<code>/api/admin/clients/{key}/toggle</code>	<code>?token=</code>	activer/désactiver
POST	<code>/api/admin/clients/{key}/delete</code>	<code>?token=</code>	supprimer

Tester au curl

bash

```
M="ton-jeton-maitre" ; A="ton-jeton-admin" ; B=http://127.0.0.1:8000
```

1. créer une clé

```
curl -s -X POST "$B/api/admin/clients?name=Pote&plan=beta&token=$A"
```

```
# → { "api_key": "as_xxx", ... } ← note la clé
```

```
K="as_xxx"
```

2. le maître publie une ouverture

```
curl -s -X POST "$B/api/signal" -H "X-Master-Token: $M" -H "Content-Type: appli"
-d '{"action":"open","ref_id":"T1","symbol":"XAUUSD","direction":"BUY","volum
```

3. le follower (clé active) récupère

```
curl -s "$B/api/signals?since=0" -H "X-API-Key: $K"
```

4. on désactive (simulate non-paiement)

```
curl -s -X POST "$B/api/admin/clients/$K/toggle?token=$A"
```

5. le follower est maintenant en pause (403)

```
curl -s "$B/api/signals?since=0" -H "X-API-Key: $K"
```

Idempotence & reconnexion (côté follower)

Le follower garde son **dernier id traité** (`since`) en local. À chaque poll il demande `?since=<dernier_id>` → pas de doublon, pas de replay. Sur un follower **neuf**: appeler d'abord `/api/status`, lire `latest_signal_id`, et partir de là (sinon il rejouerait tout l'historique).

La suite (pas encore inclus)

1. **Hook maître** dans `scalp_bot`: après chaque `order_send` réussi (ouverture et fermeture), un `POST /api/signal`. Volume envoyé en **ratio du capital**, pas en lot fixe, pour que chaque follower adapte à son compte.

2. **Bot follower** : poll `/api/signals`, applique les nouveaux signaux sur son MT5, avec **tolérance d'écart de prix max** (skip si le prix a trop bougé — clé pour ne pas diverger sur du scalp), et dédup via `since`.
3. **Stripe** : un webhook remplacera le toggle manuel → bascule `active` selon le paiement. Le reste du serveur ne bouge pas.

⚠ Avant d'exposer en ligne

MVP de dev : auth par jetons simples, HTTP local. Avant toute mise en ligne : HTTPS (reverse proxy), jetons forts en variables d'env, et durcissement de l'accès admin. Ne pas exposer `/admin` publiquement sans ça.